INFORMATIONSBLATT

IB01 · PHP-Datenbankanbindung mit PDO

LF10b · LS06 – Webanwendung mit PHP anbinden · IB01 – PHP-Datenbankanbindung · IT-3B · 2025/2026

1 · PHP auf Ubuntu 24.04

1.1 | Installation

Installation und Sicherheitseinstellungen finden Sie in der Schritt-für-Schritt-Anleitung

2 · PDO – PHP Data Objects

📖 **PDO – PHP Data Objects**
PDO ist eine PHP-Datenbankschnittstelle, die mit verschiedenen Datenbanksystemen (MySQL/MariaDB, SQLite, PostgreSQL, …) arbeitet. Der Code bleibt weitgehend gleich, egal welche Datenbank verwendet wird. PDO unterstützt **Prepared Statements** für sicheren Datenbankzugriff.

Merkmal	PDO	mysqli (direkt)
Datenbanken	Viele (MySQL, SQLite, …)	Nur MySQL/MariaDB
Prepared Statements	Ja – einfach und konsistent	Ja – etwas umständlicher
Empfohlen für	Neue Projekte, sicherer Code	Legacy-Code, MySQL-spezifisch

3 · Datenbankverbindung mit PDO aufbauen

3.1 | Verbindungskonfiguration auslagern

PHP dient in diesem Beispiel als „Zwischenebene" zwischen dem Kontaktformular (Benutzereingaben) und der Datenbank. Weil hier ungeprüfte Eingaben auf die Datenbank treffen, sind Sicherheitsmaßnahmen notwendig.

Zugangsdaten gehören in eine separate Datei `config.php` – nicht direkt in jeden PHP-Script-Code. So müssen Passwörter nur an einer Stelle geändert werden.

`/var/www/html/netconsult/config.php`

```php
<?php
define('DB_HOST', 'localhost');
define('DB_NAME', 'netconsult_db');
define('DB_USER', 'nc_web');
define('DB_PASS', 'SicheresPasswort!2025');
define('DB_CHARSET', 'utf8mb4');

function getDB(): PDO {
    $dsn = 'mysql:host=' . DB_HOST
         . ';dbname=' . DB_NAME
         . ';charset=' . DB_CHARSET;
    $options = [
        PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
        PDO::ATTR_EMULATE_PREPARES   => false,
    ];
    return new PDO($dsn, DB_USER, DB_PASS, $options);
}
```

PDO-Option	Wert	Bedeutung
ATTR_ERRMODE	ERRMODE_EXCEPTION	Datenbankfehler als PHP-Exception – kein stilles Scheitern
ATTR_DEFAULT_FETCH_MODE	FETCH_ASSOC	Ergebnisse als assoziatives Array (Spaltenname als Schlüssel)
ATTR_EMULATE_PREPARES	false	Echte Prepared Statements (serverseitig) – kein Emulieren

> 💡 **DSN – Data Source Name**
> Der DSN teilt PDO mit, welche Datenbank verwendet werden soll:
> `mysql:host=localhost;dbname=netconsult_db;charset=utf8mb4`
> Format: `treiber:host=…;dbname=…;charset=…`

4 · SQL-Injection und Prepared Statements

> 📖 **SQL-Injection**
> SQL-Injection ist einer der häufigsten Webangriffe. Dabei schleust ein Angreifer über Formulareingaben schädlichen SQL-Code ein, der die ursprüngliche SQL-Abfrage manipuliert.

Unsicherer Code – SQL-Injection möglich!

```php
<?php
// Angreifer gibt als Name ein: ' OR '1'='1
$name = $_POST['name'];  // Eingabe wird direkt verwendet!
$sql  = "SELECT * FROM kontakt WHERE name = '$name'";
```

```
// Resultierende SQL-Abfrage:
// SELECT * FROM kontakt WHERE name = '' OR '1'='1'
// → gibt ALLE Einträge zurück!
$result = $pdo->query($sql);
```

Sicherer Code – Prepared Statement mit Platzhaltern

```
<?php
// Prepared Statement: ? ist ein Platzhalter
$stmt = $pdo->prepare('SELECT * FROM kontakt WHERE name = ?');
// execute() übergibt die Werte sicher – getrennt vom SQL-Code
$stmt->execute([$_POST['name']]);
$result = $stmt->fetchAll();
// Egal was der Angreifer eingibt – es wird als normaler Text behandelt
```

⚠️ **Immer Prepared Statements verwenden!**
Prepared Statements trennen SQL-Code und Daten vollständig. Der Parameter wird nie als SQL interpretiert – egal was der Benutzer eingibt. Das gilt für **alle** Operationen: SELECT, INSERT, UPDATE, DELETE.

5 · CRUD mit PDO – Vollständiges Beispiel

INSERT – Neuen Datensatz speichern

```
<?php
require_once 'config.php';

$pdo  = getDB();
$stmt = $pdo->prepare(
    'INSERT INTO kontakt (name, email, nachricht) VALUES (?, ?, ?)'
);
$stmt->execute([
    $_POST['name'],
    $_POST['email'],
    $_POST['nachricht']
]);
$neue_id = $pdo->lastInsertId();
echo "Gespeichert mit ID: $neue_id";
```

SELECT – Datensätze lesen und anzeigen

```
<?php
require_once 'config.php';

$pdo   = getDB();
$stmt  = $pdo->query('SELECT * FROM kontakt ORDER BY erstellt_am DESC');
$liste = $stmt->fetchAll();  // FETCH_ASSOC: Array mit Spaltennamen
```

```php
foreach ($liste as $eintrag) {
    echo htmlspecialchars($eintrag['name']);  // htmlspecialchars: XSS-Schutz!
    echo htmlspecialchars($eintrag['nachricht']);
}
```

💡 **htmlspecialchars() – XSS-Schutz**
XSS (Cross-Site Scripting) ist ein Angriff, bei dem Schadcode in HTML eingeschleust wird. `htmlspecialchars($text)` wandelt Sonderzeichen (`<`, `>`, `&`) in sichere HTML-Entities um. Immer vor der Ausgabe von Benutzerdaten anwenden!

Benutzereingaben validieren

```php
<?php
$name      = trim($_POST['name'] ?? '');
$email     = trim($_POST['email'] ?? '');
$nachricht = trim($_POST['nachricht'] ?? '');

// Pflichtfelder prüfen
if (empty($name) || empty($email) || empty($nachricht)) {
    $fehler = 'Bitte alle Felder ausfüllen.';
}

// E-Mail-Format prüfen
if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
    $fehler = 'Ungültige E-Mail-Adresse.';
}
```

6 · Fehlerbehandlung

try-catch für Datenbankfehler

```php
<?php
require_once 'config.php';

try {
    $pdo  = getDB();
    $stmt = $pdo->prepare('INSERT INTO kontakt (name, email, nachricht) VALUES (?, ?, ?)');
    $stmt->execute([$name, $email, $nachricht]);
    $erfolg = 'Ihre Anfrage wurde gespeichert.';
} catch (PDOException $e) {
    // Fehlermeldung NICHT direkt ausgeben (gibt Datenbankstruktur preis)
    error_log('DB-Fehler: ' . $e->getMessage());
    $fehler = 'Datenbankfehler. Bitte versuchen Sie es später erneut.';
}
```

✅ **Fehlermeldungen nie direkt ausgeben!**

Datenbankfehlermeldungen enthalten Tabellennamen, Spaltennamen und SQL-Struktur. Diese Informationen helfen Angreifern. Interne Fehler in die Apache-Logdatei schreiben (`error_log()`), dem Benutzer nur eine allgemeine Meldung zeigen.